

Производительность GC-Agro-Mobile: предложения по исправлению

Основание: аудит производительности мобильного приложения по итогам спринта 1 (профилирование и оптимизация React Native).

Предмет: конкретные исправления, приоритеты и порядок работ. Отдельно выделено, что требует замера до изменений.

Краткое резюме

- Аудит выявил несколько мест, где тяжёлая работа выполняется на каждом рендере: пересборка GeoJSON карты, сортировка тысяч строк, отрисовка длинных списков без виртуализации.
- Обнаружены и обратные случаи - мемоизация применена там, где операция дешёвая. Это не «на всякий случай», а лишние затраты памяти и сравнений.
- Предлагается не чинить всё сразу, а взять три критичных места, подтвердить проблему замером профайлера и исправить поочерёдно.
- Первым берётся экран реестров (RegistersScreen): наибольший эффект, минимальный риск, изолированные изменения.

Ключевой принцип работы: сначала замер, потом правка, потом повторный замер. Оптимизация без подтверждения профайлером - это угадывание.

1. Порядок работы

Для каждого исправления применяется один и тот же цикл, отработанный на учебном проекте в рамках спринта:

Шаг	Действие	Инструмент	Результат
1	Снять профиль проблемного экрана	React DevTools Profiler	число перерисованных узлов и время рендера «до»
2	Зафиксировать плавность	Perf Monitor / Frame Stats	FPS при скролле «до»
3	Внести исправление	-	изменение по одной причине за раз
4	Снять профиль повторно	те же инструменты	цифры «после» для сравнения
5	Оформить PR	-	описание проблемы, решения и замеров

Замеры, приведённые в аудите как оценка «FPS 5-15», получены анализом кода, а не измерением. До внесения правок эти значения необходимо подтвердить профайлером - иначе есть риск оптимизировать то, что не является узким местом.

2. Приоритеты исправлений

Находки разделены по реальному влиянию на пользователя, а не по тому, как «неаккуратно» выглядит код.

Приоритет	Проблема	Эффект	Риск правки
Critical	Экран реестров: 5000 строк в ScrollView, сортировка на каждом рендере	фриз при открытии и скролле	низкий
Critical	useFieldsGeoJsonForMap: GeoJSON пересобирается на каждый рендер	лаги при пане и зуме карты	низкий
Critical	mapCache: сборка и кодирование стиля карты на каждом рендере	блокировка JS-потока	средний

Приоритет	Проблема	Эффект	Риск правки
Major	Длинные списки без React.memo и стабильных обработчиков	перерисовка всего списка при выборе элемента	низкий
Major	Отсутствие дебаунса в поиске и фильтрах	фильтрация на каждую введённую букву	низкий
Minor	Инлайн-стили вместо StyleSheet в редко рендерящихся экранах	практически нет	низкий
Minor	Лишняя мемоизация дешёвых операций	незначительные накладные расходы	низкий

3. Исправление 1. Экран реестров (RegistersScreen)

Проблема

- Список из тысяч строк рендерится в ScrollView - все элементы создаются сразу, включая невидимые.
- Сортировка выполняется прямо в теле компонента, то есть повторяется на каждом рендере.
- Элементы списка не мемоизированы, обработчики создаются инлайн - при выборе одной строки перерисовывается весь список.

Решение

- Заменить ScrollView на FlatList с виртуализацией: initialNumToRender, windowSize, removeClippedSubviews.
- Вынести сортировку в useМемо с зависимостью от исходных данных и параметров сортировки.
- Вынести элемент списка в отдельный компонент под React.memo.
- Обработчик выбора обернуть в useCallback, чтобы React.мемо действительно работал (иначе новая ссылка на каждый рендер сбрасывает мемоизацию).
- Вынести renderItem из JSX, чтобы он не пересоздавался.

```
// было const sorted = data.sort(...); // на каждый рендер
<ScrollView>{sorted.map(...)}</ScrollView> // стало const sorted = useМемо(() => [...data].sort(...),
[data, sortKey]); const handleSelect = useCallback((id) => setSelected(id), []); const renderItem =
useCallback(({ item }) => ( <RegisterRow item={item} isSelected={item.id === selected}
onSelect={handleSelect} /> ), [selected, handleSelect]); <FlatList data={sorted} keyExtractor={(i)
=> String(i.id)} renderItem={renderItem} initialNumToRender={15} windowSize={10}
removeClippedSubviews />
```

getItemLayout добавлять только если высота строки фиксирована и известна заранее. При переменной высоте эта оптимизация неприменима и нарушит позиционирование скrolла.

Ожидаемый результат. Отрисовка только видимых строк вместо всех; при выборе строки перерисовываются единицы элементов вместо всего списка. Подтверждается сравнением профилей до и после.

4. Исправление 2. Сборка GeoJSON для карты

Проблема

Хук useFieldsGeoJsonForMap собирает GeoJSON полей, включая разбор геометрии, но результат не закреплён за исходными данными. В результате тяжёлая сборка повторяется при каждом рендере карты, то есть при каждом пани и зуме.

Решение

- Мемоизировать сборку GeoJSON с корректным массивом зависимостей: пересборка должна происходить только при изменении набора полей, а не при любом рендере.
- Разбор геометрии (JSON.parse) выполнять один раз при загрузке данных, а не внутри рендер-цикла.
- Обработчики движения карты (пан, зум) не должны запускать пересчёт данных; при необходимости - применить дебаунс.

```
const geoJson = useМемо( () => buildGeoJson(fields), // тяжёлая сборка [fields]
// пересобирается только при смене данных );
```

В учебном эксперименте мемоизация сборки GeoJSON из 3000 точек не дала выигрыша - операция оказалась дешёвой. В боевом коде данные другие: поля представлены полигонами с сотнями координат, и разбор геометрии выполняется для тысяч объектов. Поэтому здесь эффект ожидается, но его также следует подтвердить замером.

5. Исправление 3. Кеш стиля карты

Проблема

Формирование URL стиля карты предполагает сборку и кодирование данных. Если это выполняется в теле компонента, операция повторяется при каждом рендере и блокирует JS-поток.

Решение

- Вынести формирование стиля за пределы рендера: выполнять один раз при инициализации либо мемоизировать по зависимостям, определяющим стиль.
- Кешировать результат между рендерами и не пересоздавать его при изменении не связанных с картой состояний.

Отдельная рекомендация аудита - мемоизировать `Object.freeze` возвращаемого объекта - не принимается. Заморозка небольшого объекта дешева, а мемоизация добавит накладные расходы. Это пример оптимизации, которая не окупается.

6. Убрать лишнюю мемоизацию

Мемоизация не бесплатна: она удерживает результат в памяти и на каждом рендере сравнивает зависимости. На дешёвых операциях она проигрывает прямому вычислению.

Что найдено	Почему лишнее	Действие
<code>useMemo</code> вокруг приведения типа (<code>String(id)</code>)	операция дешевле сравнения зависимостей	убрать
<code>useCallback</code> для функции, которая не передаётся в дочерний компонент и не входит в зависимости	стабильная ссылка никому не нужна	убрать
Мемоизация примитивов	примитивы сравниваются по значению	убрать

Критерий. Мемоизация оправдана в двух случаях: операция реально дорогая, либо стабильная ссылка требуется для работы `React.memo` или зависимостей хука. Во всех остальных случаях она вредит.

7. Дебаунс для поиска и фильтров

Там, где фильтрация или запрос запускаются на каждое изменение ввода, следует применять отложенный запуск: значение поля обновляется мгновенно, а тяжёлая операция выполняется один раз после паузы.

```
const [query, setQuery] = useState(""); const [debounced, setDebounced] = useState(""); useEffect(() => { const timer = setTimeout(() => setDebounced(query), 300); return () => clearTimeout(timer); }, [query]); const filtered = useMemo(() => filter(data, debounced), [data, debounced]);
```

Уточнение. Дебаунс не ускоряет саму фильтрацию - он сокращает частоту её запусков. Это прямо отвечает требованию к приложению: интерфейс не должен подвисать при вводе.

8. Что не следует исправлять сейчас

- Инлайн-стили в экранах, которые рендерятся редко и не участвуют в скролле: замена на `StyleSheet` не даст измеримого эффекта.
- Экраны-заглушки (например `LayersScreen`): их предстоит переписать, оптимизировать текущую реализацию нецелесообразно.
- Мемоизация `Object.freeze` и подобных дешёвых операций - см. раздел 6.

Обоснование. Смешивать косметические правки с оптимизацией вредно: изменения становится трудно проверить замером, а PR - трудно ревьюить. Нарушения дизайн-системы (хардкод цветов и отступов) фиксируются отдельно как технический долг.

9. Предлагаемый план работ

Шаг	Работа	Проверка	Оценка
1	Снять профили трёх критичных мест: реестры, карта, стиль карты	цифры «до» зафиксированы	0.5 дня
2	Исправить <code>RegistersScreen</code> (<code>FlatList</code> , <code>memo</code> , <code>useCallback</code> , <code>useMemo</code> для сортировки)	профиль «после», сравнение	1 день
3	Исправить <code>useFieldsGeoJsonForMap</code> (зависимости <code>useMemo</code> , разбор геометрии вне рендера)	замер пана и зума карты	0.5 дня

Шаг	Работа	Проверка	Оценка
4	Вынести сборку стиля карты из рендера	замер старта карты	0.5 дня
5	Убрать лишнюю мемоизацию	код-ревью, регрессия не ожидается	0.5 дня
6	Добавить дебаунс в поиск и фильтры	профиль ввода: один тяжёлый коммит вместо серии	0.5 дня

Каждый шаг оформляется отдельным PR с замерах «до» и «после». Один PR - одна причина изменения. Так результат можно проверить, а при регрессии - откатить точно.

10. Критерии готовности

- Открытие экрана реестров не вызывает фриза; при выборе строки перерисовываются единицы элементов, а не весь список.
- Пан и зум карты не приводят к пересборке GeoJSON (подтверждается профилем).
- Ввод в поле поиска не запускает фильтрацию на каждую букву.
- Мемоизация присутствует только там, где она обоснована замером или необходимостью стабильной ссылки.
- Каждое изменение сопровождается сравнением профилей до и после.